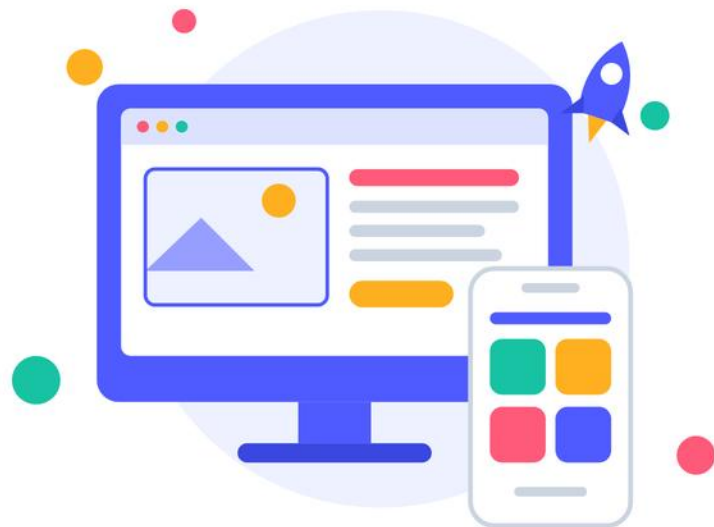


WEEK 01 · WEB PLATFORM · HTML · GIT

WHAT THE BROWSER ACTUALLY DOES

You already program. Today you find out what happens to the text you send, and how to never lose a version of it again.

YOSSI ARVIV · FRONT-END WEB DEVELOPMENT · RUPPIN



Every great developer you know got there by solving problems they were unqualified to solve until they actually did it.

PATRICK MCKENZIE

WHERE WE ARE

Week 1 of 12. Nothing to build on yet — this is the ground floor.

AFTER TODAY YOU CAN

Write a semantic, accessible, valid page from an empty file, and put it on GitHub with a history worth reading.

WHAT THIS UNLOCKS

Your repo goes live today and every week lands in it. Today's page is week 2's starting point — and the project's, in week 6.

ACT 1 · THE PROBLEM

YOU PRESS ENTER. THEN WHAT?

Between the keypress and the first pixel, at least seven separate things happen — on at least three different machines.

Name them. You have thirty seconds and a neighbour.

You have done this ten thousand times and never once been told what it does. That is not a gap in your knowledge — it is a gap in how programming is taught.

THE WHOLE TRIP, ONCE

1 Parse the URL

scheme, host, path — `https://ruppin.ac.il/courses`

2 DNS

host name becomes an IP address

3 TCP + TLS

open a connection, agree on encryption

4 HTTP request

a few lines of plain text asking for one thing

5 HTTP response

status, headers, and a body that is also plain text

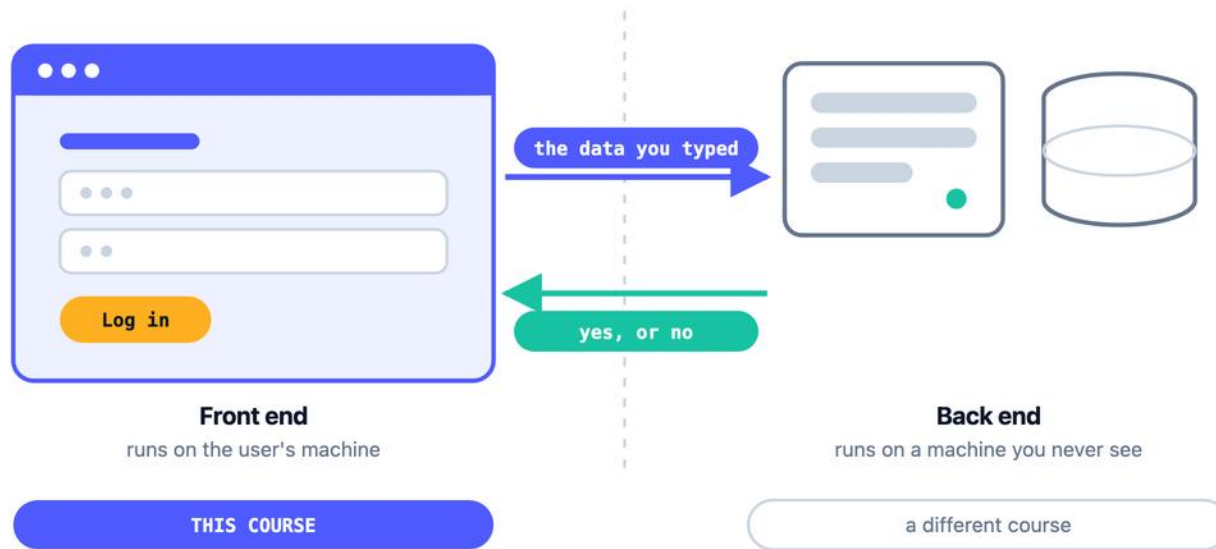
6 Parse

text becomes the DOM tree; CSS becomes the CSSOM

7 Render

layout, paint, composite — pixels

FRONT END, BACK END



FRONT END — THIS COURSE

Everything that runs **in the browser, on the user's machine**. HTML, CSS, JavaScript. You can read all of it: press `Ctrl/Cmd+U` on any site and there it is.

BACK END — NOT THIS COURSE

Runs **on the server**: databases, accounts, payments. You never see the code. It answers your requests and it does not trust you.

DNS



DNS works like a phonebook. Without DNS, you'd have to memorize numbers instead of names.

`ruppin.ac.il` is for humans. A connection is opened to an **address**, so before anything else the browser has to ask someone what that address is.

► **TRY IT LIVE** — `ping ruppin.ac.il` in the terminal. One second, and the name becomes a number in front of the room.

ONE TRIP, TWO MESSAGES



THE S IS NOT DECORATION

HTTP://

Every byte travels in the clear. Anyone between you and the server — the café wifi, the network you are on — can read it, and change it.

HTTPS://

The same HTTP, inside TLS. Encrypted in transit, and the certificate proves the server is who it claims to be.



The padlock in the address bar means the connection is private — not that the site is honest. Both are worth saying out loud.

REQUEST AND RESPONSE ARE JUST TEXT

WHAT YOU SEND

```
1 GET /courses HTTP/1.1
2 Host: ruppun.ac.il
3 Accept: text/html
4 User-Agent: Mozilla/5.0 ...
```

WHAT COMES BACK

```
1 HTTP/1.1 200 OK
2 Content-Type: text/html; charset=utf-8
3 Content-Length: 1841
4
5 <!doctype html>
6 <html lang="he">
7 ...
```

GET read · **POST** create · **PUT** replace · **DELETE** remove. **200** fine · **301** moved · **404** no such thing · **500** the server broke.



MARKUP IN, TREE OUT

Type on the left – the DOM rebuilds on the right

↺ reload

open ↗

1 — THE TEXT YOU SEND

reset

Edit this. The tree on the right is rebuilt as you type.

```
<article>
  <h2>Trip report</h2>
  <p>We walked <strong>17 km</strong>.</p>
  
</article>
```

2 — THE TREE THE BROWSER BUILT

```
article Element
  h2 Element
    "Trip report" Text
  p Element
    "We walked" Text
    strong Element
      "17 km" Text
    "." Text
  img src="map.svg" alt="Route map" Element
```

Every node is an object with a parent, children and properties. You have written this data structure before — it is a tree, and you traversed one in Data Structures.

► **HANDS-ON** [demos/02-dom-tree/](#) — Type in the left pane. Then delete one `</p>` and watch the tree change shape.

THE DOM IS A TREE. YOU HAVE WRITTEN ONE.

C#, SECOND YEAR

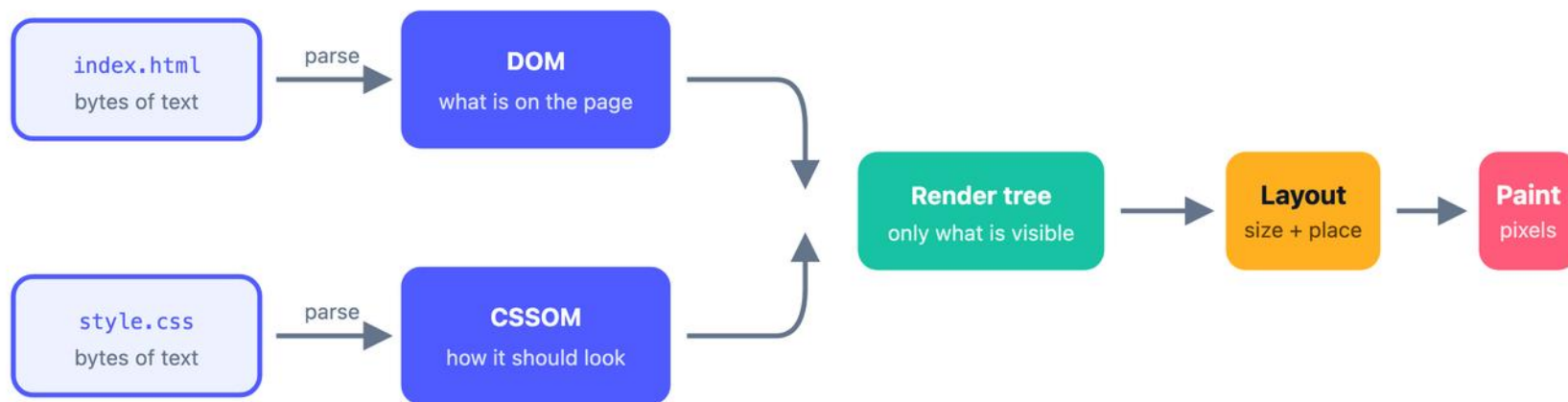
```
1 class Node {  
2     public string Tag;  
3     public Node Parent;  
4     public List<Node> Children;  
5 }  
6  
7 // walk it  
8 foreach (var c in node.Children) ...
```

THE DOM, TODAY

```
1 element.tagName;    // "ARTICLE"  
2 element.parentNode; // the node above  
3 element.children;   // the nodes below  
4 element.textContent; // the text inside  
5  
6 // walk it  
7 for (const c of element.children) ...
```

The DOM is a tree-like structure created by the browser that represents the HTML elements as JavaScript objects.

TWO TREES, THEN PIXELS



display:none never reaches the render tree — it has no box to lay out.

CSS blocks rendering on purpose. A page painted before its stylesheet arrived would flash unstyled, and you would call that a bug.

DEVTOOLS SHOWS THE TREE, NOT YOUR FILE

ELEMENTS

The *live* DOM. Edit here and the page changes; reload and it is gone.

NETWORK

Every request, its status and its size. This is where 404s confess.

ELEMENTS IS NOT YOUR FILE

```
<!-- what you wrote -->  
<p>Trip report  
  
<!-- what Elements shows -->  
<p>Trip report</p>
```

Six words for the whole pipeline: request, response, parse, style, layout, paint.

You never typed that closing tag. The browser did, while building the tree — and the tree is what Elements is a window onto.

ACT 2 · THE PROBLEM

THE SMALLEST PAGE THAT ACTUALLY WORKS

Six lines. Four of them exist for reasons you cannot see by looking at the result — and every one of those four breaks something specific when it is missing.

Which four? And what does each one break? Hold your answer — you will watch three of them break in the next ten minutes.

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="utf-8">
    <title>Maya Cohen</title>
  </head>
  <body></body>
</html>
```

HTML IS NOT A PROGRAMMING LANGUAGE

HTML is the skeleton of every web page.

Not a programming language → it's a markup language.

There is no `if`, no loop, no variable. You are not telling the browser what to do. You are telling it **what things are**.

THE SKELETON, LINE BY LINE

```
1 <!doctype html>
2 <html lang="en">
3   <head>
4     <meta charset="utf-8" />
5     <meta name="viewport"
6       content="width=device-width, initial-scale=1" />
7     <title>Maya Cohen</title>
8   </head>
9   <body>
10    <h1>Maya Cohen</h1>
11  </body>
12 </html>
```

CHARSET – THE LINE THAT DECIDES IF HEBREW WORKS

MISSING

```
1 <head>
2   <title>קורות חיים</title>
3 </head>
```

The browser guesses the encoding. On the wrong guess you get:

×§×•×"×•×@ ×–×™×™×

DECLARED

```
1 <head>
2   <meta charset="utf-8" />
3   <title>קורות חיים</title>
4 </head>
```

Bytes are read as UTF-8. Hebrew, emoji and — all survive.

VIEWPORT — THE LINE THAT DECIDES IF PHONES WORK

MISSING

```
1 <head>
2   <meta charset="utf-8" />
3 </head>
```

The phone pretends to be a 980px desktop and shrinks the whole page. Text becomes unreadable and your media queries never fire.

DECLARED

```
1 <head>
2   <meta charset="utf-8" />
3   <meta name="viewport"
4         content="width=device-width,
5               initial-scale=1" />
6 </head>
```

1 CSS pixel = 1 device-independent pixel. The page is laid out for the real screen.

ELEMENTS NEST. THEY NEVER OVERLAP.

CROSSED

```
<p><strong>Careful</p></strong>
```

NESTED

```
<p><strong>Careful</strong></p>
```

Close elements in the reverse order you opened them (LIFO).

You know LIFO. It is the same stack you push and pop. The browser keeps one while it parses, and it will silently repair yours — into something you did not write.

SOME ELEMENTS HAVE NOTHING TO CLOSE

A **void element** has no content, so it has no closing tag. Everything it says, it says in its attributes.

`</img>` and `</br>` are not "optional" — they are not a thing. Write one and the browser discards it, silently, like everything else.

```
1 
2 <br>
3 <hr>
4 <input type="email" name="email">
```

Four you will meet this week. Two of them — `br` and `hr` — you will meet again on the next slide, being used wrongly.

THE WHOLE VOCABULARY, ON ONE PAGE

DOCUMENT

html · head · body · title · meta ·
link · script · style

LISTS

ul · ol · li · dl · dt · dd

FORMS

form · fieldset · legend · label ·
input · textarea · select ·
option · button

SECTIONING

header · nav · main · section ·
article · aside · footer · h1–h6

LINKS AND MEDIA

a · img · picture · source ·
figure · figcaption · video ·
audio · svg · iframe

INTERACTIVE

details · summary · dialog ·
progress · meter

TEXT

p · strong · em · b · i · small ·
mark · time · code · pre ·
blockquote · abbr

TABLES

table · caption · thead · tbody ·
tfoot · tr · th · td

MEANINGLESS ON PURPOSE

div · span — and that is the entire point of
them.

About 110 elements exist. These are the ones you will actually use, and you already know what most of them mean. **This page is in the reference deck too — keep it open.**

LIVE CODING · BREAK 1

FROM AN EMPTY FILE

demos/01-first-page — this is where we arrive

↻ reload

open ↗

Maya Cohen

Third-year student. Learning to build for the web.

- [About me](#)
- [Projects](#)

About me

I already write C# and Python. This semester I am finding out what the browser actually does with the text I send it.

Projects

- A page that validates clean — this one.
- Whatever week 2 turns it into.

Built in class, from an empty file, in about eight minutes.

► **HANDS-ON** demos/01-first-page/ — Open 01-blank/ in Live Server FIRST — a white rectangle — then type the skeleton live.

Watch. Do not type along yet — you will build your own in the lab.

ACT 3 · THE PROBLEM

TWO PAGES. IDENTICAL PIXELS.

Press "show landmarks"

↶ reload

open ↷

Ruppín Coffee Club

[Opening hours](#) [This week's beans](#)

Opening hours

Sunday to Thursday, 08:00–16:00. Closed on exam days, obviously.

This week's beans

Ethiopia, Guji. Tastes of stone fruit and of someone else paying.

Trivia: the first webcam watched a coffee pot in Cambridge.

Built by second-years who should have been revising.

show landmarks

Press the button.

► **HANDS-ON** `demos/03-semantics/` — Press `show landmarks` here, then open `02-semantic/` in a second tab and press it there.

THE FOUR YOU WILL USE ON EVERY PAGE

- `<header>`** Introductory content for the page or for a section. Usually the title and the nav.
- `<nav>`** A block of *major* navigation links.
- `<main>`** The one region unique to this page. **Exactly one per document.**
- `<footer>`** Closing content: author, copyright, related links.

Not every link list is a nav.

A NAV IS A LIST OF LINKS, AND LISTS NEST

THE ONE THE ASSIGNMENT ASKS FOR

```
1 <nav aria-label="Main">
2   <ul>
3     <li><a href="#about">About me</a></li>
4     <li><a href="#work">Projects</a></li>
5     <li><a href="contact.html">Contact</a></li>
6   </ul>
7 </nav>
```

`nav` > `ul` > `li` > `a`. Every link is a list item; nothing else goes between them.

ORDERED, AND NESTED

```
1 <ol>
2   <li>Install VS Code
3     <ul>
4       <li>macOS: drag to Applications</li>
5       <li>Windows: run the installer</li>
6     </ul>
7   </li>
8   <li>Install Node</li>
9 </ol>
10
11 <ol start="7"><li>...continues at 7</li></ol>
```

A nested list goes **inside an `li`**, never between two of them. `ol` when the order is a fact; `ul` when it is not.

SECTION, ARTICLE, ASIDE

- `<section>`** A thematic group of content. It should have a heading.
- `<article>`** Something that stands on its own: a post, a card, a review, a comment.
- `<aside>`** Tangentially related — a sidebar, a pull quote, a "did you know".
- `<div>`** No meaning at all. A hook for styling. Still legitimate — just honest.

This content should be meaningful on its own, even if separated from the rest of the webpage.

DIV AND SPAN MEAN NOTHING — ON PURPOSE

<DIV> — A BLOCK

Takes the full width and starts a new line. A box you group things into.

 — INLINE

Sits inside a line of text without breaking it. For a few words, not a region.

```
1 <div class="price-box">
2   Total: <span class="currency">£</span>149
3 </div>
4
5 <!-- neither says what the content IS. That is the whole point. -->
```

Both are legitimate. Reach for them when there is genuinely no meaning to express — and reach for `section` or `article` when there is.

THREE RULES THAT WILL SAVE YOU MARKS

Avoid using `<section>` purely for styling — use `<div>` for that.

Headings are not for styling text big/bold.

Link text should describe the destination ("View pricing"), not "Click here".

H1 IS AN OUTLINE LEVEL, NOT A FONT SIZE

CHOSEN BY SIZE

```
1 <h1>My site</h1>
2
3 <h4>About me</h4>
4 <p>...</p>
```

"h4 because h2 looked too big."

CHOSEN BY STRUCTURE

```
1 <h1>My site</h1>
2
3 <h2>About me</h2>
4 <p>...</p>
5
6 /* next week, in CSS: */
7 h2 { font-size: 1.25rem; }
```


 IS NOT SPACING

FAKED WITH BR

```
1 Opening hours<br>
2 <br>
3 Sunday to Thursday<br>
4 08:00–16:00
```

REAL ELEMENTS

```
1 <h2>Opening hours</h2>
2 <p>
3   Sunday to Thursday<br />
4   08:00–16:00
5 </p>
```

`<br>` means "this line break is part of the content" — an address, a poem, a postal label. It is not a way to push things apart.

 LOOKS BOLD. IS IMPORTANT.

PRESENTATION

```
<p>Deadline: <b>Sunday</b></p>  
<p><i>Do not be late</i></p>
```

MEANING

```
<p>Deadline: <strong>Sunday</strong></p>  
<p><em>Do not be late</em></p>
```


Strong importance. A screen reader can change tone.

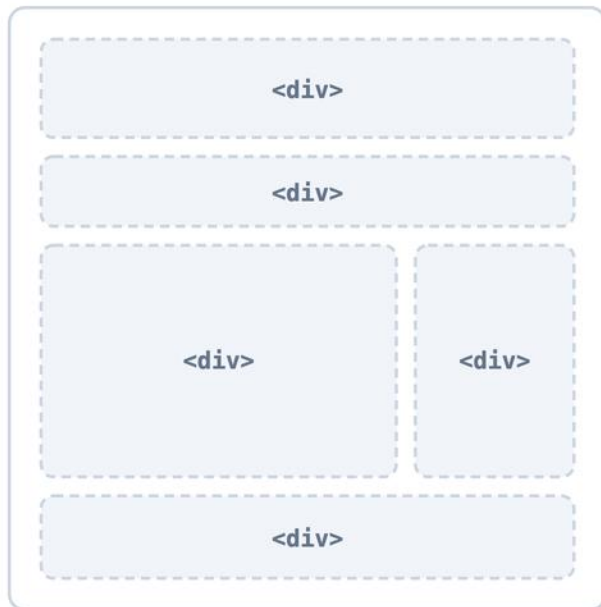
Stress emphasis — the word that changes the sentence's meaning.

 <i>

Bold and italic with no added meaning. A product name, a taxonomic term.

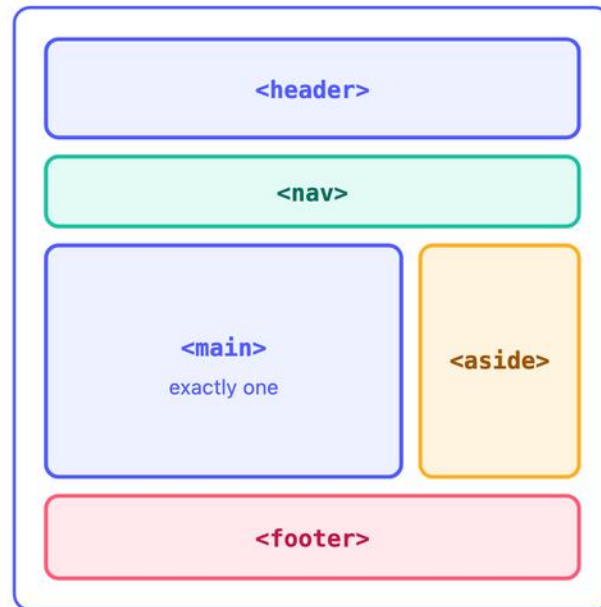
SAME PIXELS, DIFFERENT KNOWLEDGE

Div soup



Landmarks a screen reader can jump to: none

Semantic



Landmarks: header · nav · main · aside · footer

identical pixels

WHAT DIV SOUP ACTUALLY COSTS YOU

Week 5

Tailwind on a div soup produces class strings nobody can read or change.

Week 8

`querySelector('main')` returns `null`. There is no main.

Week 13

In the exam you get 90 minutes on your own project. You will navigate it by structure, under time pressure, with no search engine.

Every week

Accessibility and structure are rubric lines from today, not from week 6.



ACT 4 · THE PROBLEM

"IT WORKS ON MY MACHINE"

The single most common way a week-1 submission arrives broken: images and links written as paths that exist on exactly one computer in the world.

WHAT I OPEN

Your page, on my laptop, from your repo. Not your Desktop. If the path only resolves on your machine, the mark is for a page with holes in it.

ONLY EVER WORKED ONCE

```
  
  

```

Three paths. Three 404s on any other computer — and the validator says nothing.

IMAGES WITHOUT ALT ARE IMAGES THAT VANISH

NO ALT

```
1 
2
3 
```

DESCRIBED, OR EXPLICITLY SILENT

```
1 
4
5 
```

`alt=""` is a decision: "this picture carries no information, skip it". A missing `alt` is not a decision — the screen reader reads the file name.

LINKS SAY WHERE THEY GO

MEANINGLESS, AND UNSAFE

```
1 <a href="/pricing">Click here</a>
2
3 <a href="https://mdn.io" target="_blank">MDN</a>
```

DESCRIPTIVE, AND SAFE

```
1 <a href="/pricing">View pricing</a>
2
3 <a href="https://mdn.io"
4   target="_blank"
5   rel="noopener noreferrer">MDN</a>
```

`target="_blank"` hands the new tab a reference back to your page.

`rel="noopener noreferrer"` takes it away.

FIVE KINDS OF HREF

```
1 <a href="https://developer.mozilla.org">MDN</a>
2 <a href="about.html">The longer version</a>
3 <a href="#contact">Jump to contact</a>
4 <a href="mailto:maya@ruppin.ac.il">Email me</a>
5 <a href="tel:+972501234567">Call me</a>
```

absolute

Another site. Full scheme and host, and it never changes meaning.

relative

A file next to this one. **The only kind that survives being moved to my machine** — this is the paths section, three slides from now.

#fragment

An `id` on this page. `href="#contact"` finds `id="contact"`. This is what your `nav` uses.

mailto:

Opens a mail client with the address filled in.

tel:

On a phone, dials. On a laptop, usually nothing — and that is fine.



THREE KINDS OF PATH, AND ONLY ONE TRAVELS



images/logo.svg
relative to this file — travels with the folder ✓

/images/logo.svg
from the SERVER root — breaks in a subfolder ✗

C:\Users\maya\Desktop\images\logo.svg
one machine on earth has this folder ✗

file:///Users/maya/Desktop/images/logo.svg
not an address on the web at all ✗

All four are valid HTML. Only the first one is a working page.

LIVE: THREE BROKEN PATHS

demos/04-paths/01-broken – valid HTML, three 404s

reload

open

Three paths, three ways to fail

1. `file:///Users/maya/Desktop/site/images/logo.svg`



An absolute path into someone's Desktop. Nobody else has that folder.

2. `c:\Users\maya\Desktop\site\images\logo.svg`



Same mistake, Windows flavour. Backslashes are not path separators on the web.

3. `/images/logo.svg`



► **HANDS-ON** demos/04-paths/ — Open DevTools > Network and reload. Three 404s. Then walk 02-fixed/ both ways.

A PAGE IS ALWAYS OPENED WITH LIVE SERVER

DOUBLE-CLICK

```
file:///Users/maya/Desktop/site/index.html
```

LIVE SERVER

```
http://127.0.0.1:5500/index.html
```

Week 7 ES modules refuse to load over `file://`.

Week 10 `localStorage` is scoped to an origin. `file://` barely has one.

Week 11 `fetch` is blocked outright.

ACT 5 · THE PROBLEM

CLICK THE WORDS "FULL NAME"

demos/05-forms/01-broken — nothing happens

↶ reload

open ↷

Contact us

Full name

Email

Message

Press "send" to see what the server would receive.

► **HANDS-ON** demos/05-forms/ — Click the words "Full name". Nothing. Press send . Nothing. Then run the validator on it — it passes.

THE FORM, DONE PROPERLY

```
1 <form action="/contact" method="post">
2   <fieldset>
3     <legend>Your details</legend>
4
5     <label for="name">Full name</label>
6     <input type="text" id="name" name="name" required />
7
8     <label for="email">Email</label>
9     <input type="email" id="email" name="email" required />
10
11    <button type="submit">Send</button>
12  </fieldset>
13 </form>
```

PLACEHOLDER IS NOT A LABEL

PLACEHOLDER ONLY

```
1 <input type="text" placeholder="Email" />
```

Disappears the moment you type. Fails contrast.
Cannot be clicked.

LABEL + FOR

```
1 <label for="email">Email</label>  
2 <input type="email" id="email" name="email" />
```

`for` points at `id`. Clicking the label focuses the field.

placeholder is not a label, always provide a real `<label>`.

TYPES DO WORK FOR YOU. NAME DECIDES WHAT IS SENT.

type

email tel number date url

password checkbox radio — each

brings its own keyboard on mobile and its own validation.

name

The key the server receives. No name, no data — the field is ignored.

id

Unique in the document. What label for points at.

required

The browser refuses to submit an empty field, and says so in the user's language.

WHAT THE SERVER ACTUALLY RECEIVES

```
<input type="email" name="email" id="a" required>  
<input type="text" name="name" id="b">  
<input type="date" id="c">
```

?

email=maya%40ruppin.ac.il&name=May

a

The third field has no name. It is on the page, the user filled it in, and it is not in that string.

LIVE CODING · BREAK 2

BREAK IT THREE WAYS

demos/06-validator/01-broken – three planted errors

↶ reload

open ↷

Trip report — Golan, March

The route

Seventeen kilometres, most of it uphill, all of it in the rain.

The photographs



[The version that validates](#)

► **HANDS-ON** demos/06-validator/ — Run `npx html-validate` on this file in the integrated terminal. Do not say what is broken.

READING THE OUTPUT

```
1 49:8 error Unclosed element '<section>' close-order
2 52:8 error Stray end tag '</div>' close-order
3 54:20 error Duplicate ID "route" no-dup-id
4 56:10 error <img> is missing required "alt" attribute wcag/h37
5 58:6 error End tag '</main>' seen but there were open elements close-order
6 63:4 error End tag '</body>' seen but there were open elements close-order
7 64:2 error End tag '</html>' seen but there were open elements close-order
```

Three mistakes, seven lines. Fix the **first** error and four of these disappear on their own.



ACCESSIBILITY IS GRADED FROM TODAY

Keyboard

Tab through the whole page. Everything interactive must be reachable.

Focus

You must be able to see where you are. Never delete the focus ring.

Landmarks

`header` `nav` `main` `footer` — how a screen-reader user skips to the content.

Names

Every image, every field, every link says what it is.

Every assignment this semester has a rubric line for this. Including this week's.

ACT 6 · THE PROBLEM

YOU ALREADY HAVE A VERSION CONTROL SYSTEM

```
index.html  
index-v2.html  
index-final.html  
index-final-FINAL.html  
index-final-FINAL-use-this-one.html
```

It is bad. Which one did you hand in, and what exactly changed between two of them?



GIT STORES SNAPSHOTS, NOT DIFFERENCES

WHAT PEOPLE ASSUME

Git saves a list of changes and replays them to rebuild a version.

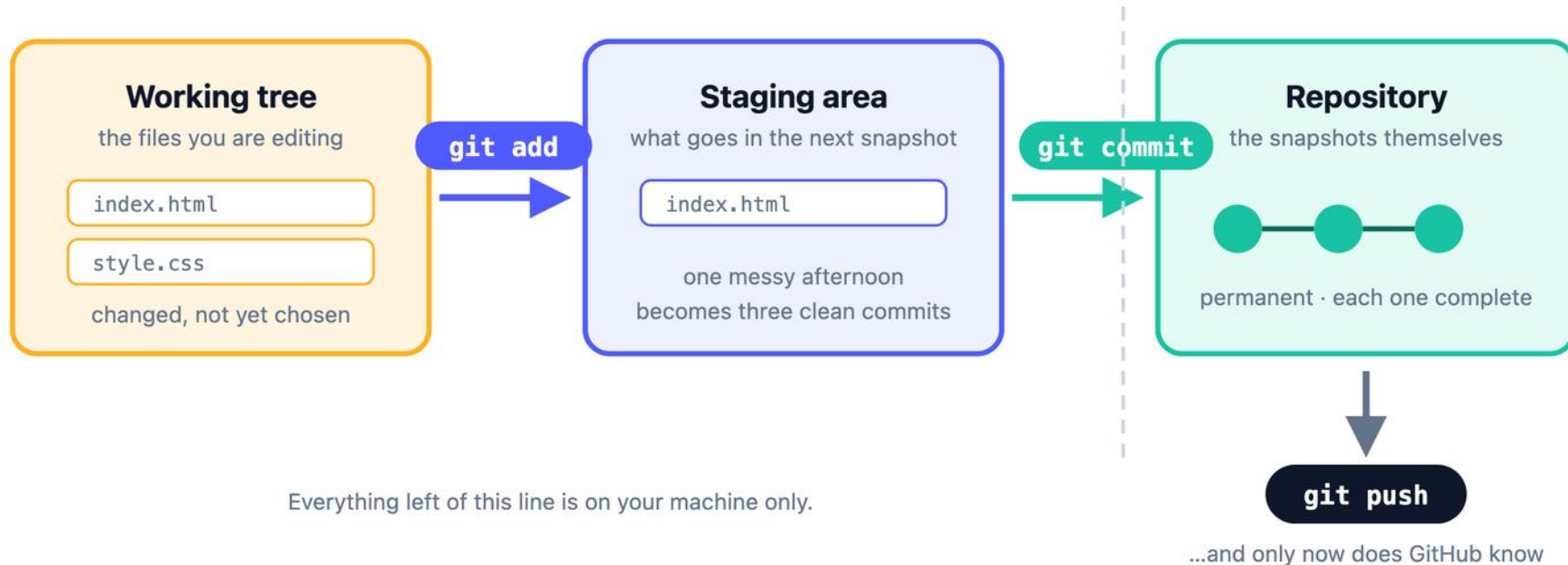
WHAT GIT DOES

Every commit is a complete picture of the project at that moment. Unchanged files are stored once and pointed at.

This is why checking out an old commit is instant, and why nothing is ever "reconstructed". The diff you read is *computed* for you, not stored.



THREE PLACES A FILE CAN BE



THE FOUR COMMANDS YOU WILL ACTUALLY RUN

WHEN YOU ARE LOST

`git status` always answers. Run it before and after everything else.

```
1 git init
2
3 git status
4
5 git add index.html
6
7 git commit -m "feat: add semantic page skeleton"
8
9 git log --oneline
```

Staging exists so one messy afternoon can become three clean commits.
Use it.

A COMMIT MESSAGE IS FOR THE PERSON READING IT IN WEEK 13

NO INFORMATION

```
update  
fix  
asdf  
final version
```

A READABLE HISTORY

```
feat: add contact form with labelled fields  
fix: correct relative path to logo  
docs: add README with run instructions  
style: format with Prettier  
refactor: extract nav into its own section
```

Small commits, often. One giant commit at the end is not a history — it is a backup.

THREE FILES THAT SHIP WITH EVERY PROJECT

README.md

What this is and how to run it. The first thing anyone opens — including me.

.gitignore

What must never be committed:

`node_modules/`, `.env`, `.DS_Store`.

Secrets in a public history stay there forever.

.gitattributes

* `text=auto eol=lf`. Already in your repo. It normalises line endings so a Mac and a Windows machine produce the same commit.

WITHOUT .GITATTRIBUTES

```
- <h1>Maya Cohen</h1>
- <p>Third-year student.</p>
+ <h1>Maya Cohen</h1>
+ <p>Third-year student.</p>
```

Two identical lines, reported as changed, because one machine ended them with `\r\n` and the other with `\n`. Do this across a whole file and nobody can review anything.

YOUR REPO, AND WHY I AM IN IT

```
1 # one PRIVATE repo for the whole semester, from the course template
2 frontend-2027-<student-id>
3
4 git clone git@github.com:<you>/frontend-2027-<student-id>.git
5
6 git add . && git commit -m "feat: week 1 personal page"
7
8 git push
```

Private

Forty public repos in one course is a copy-paste machine.

Collaborator

Add me in week 1, not at the end. In week 13 I pre-load your own project onto the exam machines — and on exam day there is no fixing this.

THE SHA FREEZES THE SUBMISSION

```
1 git log -1 --format=%H
2 9f4c1a7e0b2d5836a1c4f7e9d2b8a6c3e5f10947
3
4 # submit BOTH, in Moodle:
5 #   https://github.com/<you>/frontend-2027-<student-id> + the SHA
```

What is in that commit is what gets marked. Pushing after the deadline does not change the grade, and there is no argument about it.

THREE QUESTIONS BEFORE YOU LEAVE

1. The browser received your HTML. Name the two trees it builds, and what it makes from them.
2. Your page and your image are in the same folder. Write the `src`. Now write the one that only works on your machine.
3. You have committed and pushed. What exactly do you paste into Moodle, and why two things?

WEEK 01 · NEXT

THE LAB, AND BEFORE NEXT WEEK

IN THE LAB, NOW

Your private repo from the template, me added as a collaborator, and the first real page of your site in `week-01/`: semantic, with a form, validating clean, three meaningful commits.

TO SUBMIT

The repo URL and the commit SHA, in Moodle. Two things, nothing else.

This page is week 2's starting point.

BEFORE NEXT WEEK

Finish the setup guide if anything is still red. Next week: CSS, and this page stops looking like a 1996 Word document.

And the sheet you were handed in the first minute — the course map, how you are graded, and the three rules. Keep it.